

**Final Project Based Assignment:
Automated Chemical Dosing System
CS3243 IoT Devices and Applications**

Department of Computer Science and Engineering
University of Moratuwa

Mudaliarachchi NS - 230415H

May 30, 2026

Contents

1	Introduction	3
2	Water Level Measurement Method	3
2.1	Configurations of the Water Supply Tank	4
2.2	Method : Hydrostatic Pressure Level Sensing Routine	4
3	Concentrated Chemical Level Estimation Method	5
3.1	Method: Dual-Verification (Mass-Computational) Level Tracking for Concentrated Chemical Estimation	5
4	Hardware Specifications and Comparative Market Analysis	8
4.1	Water Level Measurement - Inferential Flow Sensors	8
4.2	Water Level Measurement - Hydrostatic Pressure Transducer	9
4.3	Chemical Level Estimation - Load Cell	10
4.4	Dispensing Tank Level Sensors	10
4.5	Flow Meters - Chemical Dosing Line	12
4.6	Control Valves - Dispensing Tank Inlets	12
4.7	Variable Rate Pump - Chemical Dosing	14
4.8	Master Controller and Slave Controllers	15
4.9	Human-Machine Interface - Digital Gauge Display	16
4.10	Audio-Visual Emergency Alarm Subsystem	17
4.11	Summary Cost Comparison	17
5	Component Failures and Automated Fail-Safes	19
5.1	Water Supply Subsystem: Hydrostatic Pressure Level Sensor Failure . . .	19
5.2	Water Supply Subsystem: Inline Flow Meter Failure	20
5.3	Chemical Dosing Subsystem: Load Cell and Weight Platform Failure . .	20
5.4	Chemical Dosing Subsystem: Actuator Failure (Pump & Solenoid Valves)	21
6	Distributed Data Acquisition and Network Communication Protocol	23
6.1	Alternative 1: Wireless MQTT Client-Broker Architecture (Rejected) . .	24
6.2	Alternative 2: Controller Area Network (CAN Bus) - Selected	24
6.3	Cloud Reporting Layer: Master Node Wi-Fi Uplink	26
7	System Architecture	27
8	Block Diagram	28
9	Operational Algorithm for IOT components	28
10	Implement simulate the solution (with one dispensing tank)	33
10.1	Documeneted Code	33
11	Schematic Diagram	42
	References	43

1 Introduction

Hospitals rely on a continuous and precisely controlled supply of disinfecting chemicals to sterilise medical instruments and maintain clinical hygiene standards. These chemicals arrive in concentrated form and must be diluted to a specific parts-per-million (PPM) concentration before being distributed to dispensing tanks located throughout the hospital premises. Any deviation from the required concentration whether through over-dilution or under-dosing poses a direct risk to patient safety and instrument sterility.

This report presents the complete design of an Internet of Things (IoT) solution for an Automated Chemical Dosing System deployed in a hospital environment. The system is engineered to satisfy the following core operational requirements:

- Continuous, non-intrusive measurement of the purified water supply tank level using a hydrostatic pressure transducer, without placing any sensor inside the tank to accommodate periodic cleaning protocols.
- Automated, on-demand refilling of up to ten dispensing tanks (750 mL each) distributed across the hospital premises, served sequentially on a first-come, first-served basis.
- Dual-verification tracking of the concentrated chemical supply level using an external load cell and a computational flow integration model, without any sensor contact with the chemical container.
- Real-time fault detection covering sensor failures, pipeline leaks, stuck-open valves, and dry-running supply tanks, with audible and visual alarms.
- A distributed controller network using a wired Controller Area Network (CAN Bus) backbone connecting one master ESP32 node and ten slave ESP32 nodes, with a Wi-Fi cloud uplink for periodic reporting.
- Periodic cloud-based reporting covering chemical consumption by tank location, daily usage totals, and refill frequency statistics.

The report is structured as follows. Section 2 presents the water level measurement methodology and the hydrostatic pressure sensing routine. Section 3 covers the concentrated chemical level estimation framework. Section 4 provides a comparative hardware market analysis for all selected components. Section 5 documents component failure modes and automated fail-safe strategies. Section 6 describes the distributed network communication protocol. Sections 7 and 8 present the system architecture and block diagram respectively. Section 9 details the operational algorithms for each IoT component. Section 10 documents the Wokwi simulation with the complete firmware. Section 11 presents the physical schematic diagram.

2 Water Level Measurement Method

Since the hospital prefers not to install any sensors inside the tank due to the periodic cleaning process, the possible way to install sensors is either on the top lid of the tank or externally at the discharge outlet point.

2.1 Configurations of the Water Supply Tank

Parameter	Value
Volume	2000 Liters
Diameter	50 cm
Height	10.16 m

Table 1: Water Supply Tank Specifications

2.2 Method : Hydrostatic Pressure Level Sensing Routine

To satisfy the hospital’s strict guidelines preventing any physical instrumentation from entering the 2000-liter purified water supply tank, a real-time Hydrostatic Pressure Level Sensing Routine is deployed within the central controller.

This model determines the continuous liquid height and net volume inside the vessel by intercepting analog voltage telemetry from a single industrial, thread-mounted pressure transducer installed completely external to the tank housing along the lowest point of the primary discharge manifold.

Structural Pressure Validation

Before sensor selection, the maximum hydrostatic gauge pressure exerted at the base of the fully-filled tank must be established to confirm that the chosen transducer’s rated range safely encompasses the worst-case operating condition.

Given the tank height of $H_{\max} = 10.16$ m and the standard volumetric density of purified water $\rho = 1000$ kg/m³:

$$P_{\max} = \rho \times g \times H_{\max}$$

$$P_{\max} = 1000 \times 9.81 \times 10.16$$

$$P_{\max} = 99,669.6 \text{ Pa} \approx \mathbf{0.0997 \text{ MPa}} \approx \mathbf{0.096 \text{ MPa}}$$

This confirms that the selected 0–10 MPa pressure transducer operates at less than **1%** of its rated full-scale capacity under maximum load, providing an extremely generous safety margin and high effective resolution within the active measurement window.

Voltage-to-Pressure Conversion

Every $\Delta t = 1.0$ second, the controller samples the ADC channel connected to the transducer. The raw analog output voltage (V_{out}) is linearised across the calibrated 0.5 V to 4.5 V range to recover the instantaneous gauge pressure:

$$P = \left(\frac{V_{\text{out}} - 0.5}{4.5 - 0.5} \right) \times 10,000,000 \quad (\text{Pascals})$$

Computing Linear Height from Pressure

By inverting the hydrostatic equilibrium, the controller derives the instantaneous linear height profile (H) of the water column directly from the recovered pressure value:

$$P = \rho \times g \times H \quad \Rightarrow \quad H = \frac{P}{\rho \times g} = \frac{P}{1000 \times 9.81} \quad (\text{meters})$$

Converting Linear Height to Net Fluid Volume

To populate the continuous digital readout gauge on the operational dashboard, the controller converts the computed height into a running volume variable using the constant cross-sectional area of the cylindrical tank:

$$r = 0.25 \text{ m}, \quad A = \pi r^2 = \pi(0.25)^2 \approx 0.19635 \text{ m}^2$$

$$V_{\text{m}^3} = H \times A$$

$$V_{\text{current}} = V_{\text{m}^3} \times 1000 \quad (\text{Liters})$$

This dynamic tracking variable continuously updates the HMI graphical dashboard, providing smooth, non-accumulating inventory tracking while allowing hospital maintenance crews to clean and manually scrub the internal tank structure without risking physical damage or fluid short-circuits to any internal instrumentation.

Leak Detection via Auxiliary Flow Meters

In addition to the primary pressure transducer, two inline SEN-HZ21WA Hall Effect turbine flow sensors are retained on the outlet pipes of the tank exclusively for anomaly detection. Under normal operation, any discrepancy between the volumetric balance computed from the flow meters and the independently measured pressure-derived volume indicates an unaccounted fluid loss event.

3 Concentrated Chemical Level Estimation Method

To adhere to the strict constraint preventing the installation of level sensors inside the 10-liter concentrated chemical container, a highly reliable, non-intrusive Dual-Verification Framework is deployed.

3.1 Method: Dual-Verification (Mass-Computational) Level Tracking for Concentrated Chemical Estimation

The system is configured around a standard 10-liter cylindrical or rectangular chemical supply vessel. The target fluid handled is industrial-grade Isopropyl Alcohol (IPA), which exhibits physical properties distinct from water.

Chemical Properties and Container Specifications

- **Fluid Density (ρ_{chem}):** At typical temperature in Sri Lanka ($27^\circ\text{C} - 32^\circ\text{C}$), the density of Isopropyl Alcohol is:

$$\rho_{\text{chem}} = 785 \text{ kg/m}^3$$

which is equivalent to:

$$0.785 \text{ g/mL} = 0.785 \text{ kg/L}$$

- **Maximum Fluid Mass:** A completely full 10-liter chemical payload exerts a net liquid mass of:

$$\begin{aligned} M_{\text{fluid}} &= \text{Volume} \times \rho_{\text{chem}} \\ M_{\text{fluid}} &= 10 \text{ L} \times 0.785 \text{ kg/L} = 7.85 \text{ kg} \end{aligned}$$

- **Container Weight:** Assume the empty plastic chemical container exhibits a static dead weight:

$$M_{\text{tare}} \approx 0.35 \text{ kg} - 0.50 \text{ kg}$$

1. Primary Physical Verification: External Load Cell Matrix

An industrial strain-gauge load cell platform is installed directly beneath the container assembly, isolating the electrical components entirely from chemical contact.

When a fresh container is positioned on the platform, the system filters out the weight of the platform frame and the container's dry plastic shell (M_{tare}).

The controller samples the raw live gross weight (M_{current}) and computes the true remaining physical volume (V_{mass}) via mass balance:

$$V_{\text{mass}} \text{ (Liters)} = \frac{M_{\text{current}} - M_{\text{tare}}}{\rho_{\text{chem}}}$$

Because the fluid density is significantly lighter than water, this equation prevents a 21.5% overestimation error that would occur if water-density constants were mistakenly applied to the software logic.

2. Secondary Verification: Computational Flow Integration Model

Operating in parallel with the weight sensor, an inferential software loop serves as a digital twin redundancy check.

As the chemical dosing pump draws fluid from the container, it passes through an inline micro-flow meter. Because microcontrollers execute code within sequential loops at finite clock edges, the theoretical continuous integral for fluid displacement is represented as:

$$V_{\text{dispensed}} = \int_0^t Q_{\text{chem}}(t) dt$$

$$\Delta t = 1.0 \text{ second}$$

The running algorithmic volume register updates dynamically every clock cycle:

$$V_{\text{algorithm}}(\text{new}) = V_{\text{algorithm}}(\text{old}) - \Delta V$$

$$\Delta V = Q_{\text{chem}} \times \Delta t$$

Where:

- Q_{chem} = Real-time volumetric flow rate extracted from the pulse frequency of the micro-flow meter (Liters/Second).
- ΔV = Absolute incremental volume drawn during the 1-second clock step (Liters).

Leak Detection Framework

The two verification channels , the physical load cell and the computational flow integration model, operate simultaneously and independently. Under normal dosing operation, both channels should agree within a configurable tolerance window. Any sustained divergence between the two measurements indicates an unaccounted fluid loss event in the chemical supply line.

4 Hardware Specifications and Comparative Market Analysis

This section presents a systematic evaluation of all hardware components required for the automated chemical dosing system. For each subsystem, three commercially available product options sourced from the Sri Lankan and foreign market are compared across cost, specifications, and suitability for the application. The recommended product for each category is highlighted in green.

4.1 Water Level Measurement - Inferential Flow Sensors

The inferential software flow integration routine requires two inline Hall Effect turbine flow sensors installed externally on the inlet and outlet pipes of the 2000-liter water supply tank. This approach eliminates any physical instrumentation inside the tank, fully satisfying the hospital’s strict cleaning protocol constraint. The total cost for two sensors is approximately **LKR 6,000**.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
SEN-HZ21WA (HT30) Hall Effect sensor	duino.lk	~LKR 3,000	1-30 L/min, 5V DC, 450 pulses/L, Nylon PA66 + stainless steel shaft, DN15	Chosen
G $\frac{1}{2}$ inch Hall sensor	duino.lk	~LKR 900	1-30 L/min, 5V DC, lower-cost plastic body	Budget alt.
YF-S201C Hall Effect Liquid Water Flow Sensor	duino.lk	~LKR 1,700	flow range 1-30 L/min, 5V DC, plastic body	Budget alt.

Table 2: Water Flow Sensor Comparison (Inferential Level Method)

Justification: The SEN-HZ21WA is selected for its stainless steel impeller shaft, which provides superior corrosion resistance and long-term durability compared to fully plastic-bodied alternatives - an important consideration given that these sensors remain permanently installed on live water pipes in a hospital environment. For the leak detection role, pulse-counting accuracy and material integrity are the primary selection criteria rather than absolute flow precision, since the sensors are used to compute a running volumetric balance that is continuously compared against the independent pressure-derived measurement. The SEN-HZ21WA’s 450 pulses-per-liter K-factor provides sufficient resolution to detect the small incremental flow discrepancies that would indicate an early-stage pipeline leak. The budget G $\frac{1}{2}$ Hall sensor, while significantly cheaper at LKR 900, uses a fully plastic body and impeller assembly that is susceptible to accelerated wear

and seal degradation under continuous flow cycling, making it unsuitable for unattended long-term deployment in a critical hospital infrastructure context.

4.2 Water Level Measurement - Hydrostatic Pressure Transducer

The hydrostatic pressure level sensing routine requires a single industrial, thread-mounted pressure transducer installed externally at the lowest point of the primary discharge manifold of the 2000-liter water supply tank. This approach eliminates any physical instrumentation inside the tank, fully satisfying the hospital’s strict cleaning protocol constraint. The maximum hydrostatic pressure at full tank ($H = 10.16$ m) is ≈ 0.096 MPa, which must be within the selected sensor’s rated range.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
0–0.5 MPa Pressure Sensor G1/4 DC 5V	duino.lk	~LKR 6,990	0–5 MPa range, G1/4 thread, 0.5–4.5 V analog output, 5V DC, $\pm 1.0\%$ FS accuracy, water/oil/gas compatible	Chosen
0–10 MPa Pressure Sensor G1/4 DC 5V	duino.lk	~LKR 9,850	0–10 MPa range, G1/4 thread, 0.5–4.5 V analog output, 5V DC, $\pm 2.0\%$ FS accuracy, water/oil/gas compatible	Over-range
G1/4 DC 5V 0–1.2 MPa Pressure Transducer	AliExpress	~LKR 11,500	0–1.2 MPa, G1/4 thread, 0.5–4.5 V output, carbon steel body, $\pm 1.5\%$ FS, ≤ 2 ms response, 5V DC	High price

Table 3: Hydrostatic Pressure Transducer Comparison (Water Level Measurement)

Justification: The 0–0.5 MPa sensor from duino.lk is selected as the optimal choice. Given the maximum operating pressure of ≈ 0.096 MPa at full tank, this sensor operates at only **1.92%** of its rated full-scale range, providing an excellent safety margin while maintaining a meaningful resolution window. The $\pm 1.0\%$ FS accuracy is tighter than the 0–10 MPa alternative ($\pm 2.0\%$ FS), which also operates at less than 1% of its range - unnecessarily compressing the effective measurement resolution for this application. The AliExpress 0–1.2 MPa transducer, while significantly not cheaper than others, operates at

8% of its rated range at full tank and uses a carbon steel body which presents long-term corrosion risk in a purified water environment. Its lower unit cost does not justify the reduced build quality and chemical compatibility risk in a critical hospital infrastructure application.

4.3 Chemical Level Estimation - Load Cell

The dual-verification framework uses a strain-gauge load cell platform mounted beneath the 10-liter IPA canister. The maximum payload is approximately 8.35 kg (7.85 kg fluid + 0.50 kg container). The load cell must accommodate this weight with adequate safety margin.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
CZL-601 60 kg S-type load cell	tronic.lk	~LKR 3,650	60 kg capacity, S-type aluminium, HX711 compatible	Suitable
20 kg load cell	duino.lk	~LKR 440	20 kg max — marginal for 8.35 kg payload; minimal overload margin	△ Risky
50 kg body load cell	duino.lk	~LKR 260	50 kg max, ample overload headroom, HX711 compatible	Best fit

Table 4: Load Cell Comparison (Chemical Canister Weighing)

Justification: The 50 kg load cell provides a 6× safety margin over the maximum payload of 8.35 kg, ensuring reliable operation without risk of overload. The 20 kg option is technically borderline and unsuitable for a hospital environment where reliability is critical. The 60 kg CZL-601 functions correctly but reduces measurement resolution since the active range (0–8.35 kg) is a smaller fraction of its full scale.

4.4 Dispensing Tank Level Sensors

Each 750 mL dispensing tank requires a single low-level binary sensor to detect when the chemical level falls below the refill threshold. The sensor is fitted externally to avoid chemical contact, and only on/off detection is required - continuous level measurement is not needed for this subsystem.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
JSN-SR04T waterproof ultrasonic sensor	lk-tronics.com	~LKR 1,690	IP67 rated, 20–450 cm range, chemical-splash resistant, sealed probe	Rec.
HC-SR04 standard ultrasonic	tronic.lk	~LKR 300	2–400 cm range, NOT waterproof, exposed PCB — chemical exposure risk	Risky
DFRobot gravity liquid sensor	dfrobot.com	~LKR 2,800	Non-contact through-wall capacitive detection, corrosion-proof, fully sealed	Alt.

Table 5: Dispensing Tank Level Sensor Comparison

Justification: The JSN-SR04T is the optimal choice. Its IP67 waterproof rating protects against chemical splashes in the dispensing environment, unlike the bare HC-SR04. The DFRobot capacitive sensor offers superior non-contact through-wall sensing and is the preferred option if the dispensing tanks are sealed plastic containers, at a higher cost premium.

4.5 Flow Meters - Chemical Dosing Line

The chemical dosing pump flow rate is controlled via a PWM signal but does not guarantee accurate flow. An inline flow meter on the chemical line measures the actual dispensed volume in real time, forming the secondary verification channel of the dual-verification framework.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
YF-S201 Hall effect flow sensor	lk-tronics.com	~LKR 900	1–30 L/min, 5V, Hall pulse output, ABS plastic body	Corrosion .
Magnetic plastic flow sensor	duino.lk	~LKR 1500	Low flow range, food-grade plastic body	alt.
YF-S201C G $\frac{1}{2}$ Hall effect sensor	duino.lk	~LKR 1,600	G $\frac{1}{2}$ thread fitting, Hall pulse output, slightly more robust pipe connection	Good fit

Table 6: Chemical Line Flow Meter Comparison

Important Note: IPA (Isopropyl Alcohol) can degrade standard NBR rubber O-rings and ABS seals over time. Before finalising any flow sensor for the chemical line, the impeller shaft material, O-ring compound, and housing material must be verified for IPA compatibility. PTFE or Viton-lined sensors are the recommended alternative for long-term deployments.

4.6 Control Valves - Dispensing Tank Inlets

Each dispensing tank requires a solenoid-controlled valve to regulate chemical inflow. The system may serve tens of dispensing tanks distributed throughout the hospital premises, making per-unit cost a significant factor.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
Tuya Zigbee smart gas valve	daraz.lk	~LKR 9,000	Zigbee wireless, smart-home integration, timer function, $\frac{1}{2}$ –1 inch	Overkill.
FPD270A $\frac{1}{2}$ inch solenoid valve (220V)	duino.lk	~LKR 1,200	220V AC, $\frac{1}{2}$ inch, normally closed, direct relay control from MCU	Best value
Fully Automatic Water Level Control Plastic Float Valve	Daraz.lk	~LKR 1,890	A 220V AC, $\frac{1}{2}$ -inch normally closed automatic float valve with PA66 nylon and 304 stainless steel construction, supporting 0.1–12 kg pressure range and direct MCU relay control	Alt.

Table 7: Control Valve Comparison (Dispensing Tank Inlets)

Justification: With potentially tens of dispensing tanks, the FPD270A at LKR 900 per valve is the clear choice. It is driven through a relay module directly from the MCU, requires no wireless protocol stack, and is purpose-built for liquid pipelines. The Tuya Zigbee valve is designed for smart-home irrigation and gas shutoff applications and would add unnecessary cost and protocol complexity to the IoT architecture.

4.7 Variable Rate Pump - Chemical Dosing

The chemical dosing pump must deliver IPA at a controllable flow rate via PWM, with the actual flow measured by the inline flow meter. Since IPA is a chemical solvent, fluid compatibility with the pump internals is the primary selection criterion.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
8W 12V peristaltic pump	Daraz.lk	~LKR 3,500	DC 12V and a power of about 8W. Operate at a measured voltage of 5-12V.	Best
Small Peristaltic Dosing Pump Micro Self-priming Mute Liquid Water Pump BPT Tubing for Lab 12V DC Pump	Daraz.lk	~LKR 5,000	DC 12V - 24V Peristaltic Liquid Micro Pump Micro Self-priming Hose Pump Head for Aquarium Lab Analytical Plate Type	High Cost
Peristaltic Pump Large Ball Bearing Liquid Dosing Pumps with Silicone Tube	Daraz.lk	~LKR 11,800	The ball bearing is in the wheel frame, which has low noise and low friction during operation, can extend the service life.	High Cost

Table 8: Chemical Dosing Pump Comparison

Justification: The peristaltic dosing pump is the architecturally correct choice for this application. In a peristaltic design, the pumped fluid travels entirely inside a replaceable tube, meaning the IPA never contacts the pump motor, housing, or impeller assembly. This eliminates chemical compatibility concerns entirely. PWM speed control is also more linear for peristaltic pumps than centrifugal designs, improving dosing accuracy. As the cost varies we can choose the 8W 12V peristaltic pump for the best choice.

4.8 Master Controller and Slave Controllers

To coordinate the real-time processing demands of the hydrostatic level tracking, high-frequency pulse integration, and automated dosing interlocks, a distributed microcontroller network is implemented. Here this system is implemented through a can bus, we need to assign each master and Slaves to the CAN bus system.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
ESP32 Development Board (38-Pin)	Duino.lk	~LKR 1,490	32-bit dual-core processor, integrated Wi-Fi and Bluetooth, 38 GPIO pins, native hardware interrupts.	Best
Arduino Uno R3 V2 Development Board	Duino.lk	~LKR 1,390	8-bit ATmega328P processor, 14 digital I/O pins, only 2 external hardware interrupt pins.	Low Perf.
Raspberry Pi 3 Model B+	Tronic.lk	~LKR 22,600	1.4GHz 64-bit quad-core ARM processor, 1GB LPDDR2 SDRAM, Linux operating system.	High Cost

Table 9: Microcontroller Selection Matrix

Justification: The ESP32 Development Board is selected as the optimal choice for both the Master and Slave processing nodes. It features a high-performance 32-bit dual-core architecture and abundant hardware interrupts, which are vital for capturing high-frequency asynchronous pulse streams from the SEN-HZ21WA flow meters without data loss.

While the Arduino Uno lacks the memory and processing cores to manage the concurrent HMI calculations alongside IoT data streaming, the Raspberry Pi 3 introduces unnecessary complexity, high power consumption, and extreme procurement costs for an embedded control application. Utilizing a uniform ESP32 architecture across both nodes simplifies inter-controller I2C/UART bus programming while providing integrated Wi-Fi capabilities for native hospital telemetry streaming.

4.9 Human-Machine Interface - Digital Gauge Display

The continuous digital readout gauge on the dashboard requires a crisp, high-contrast display module capable of rendering dynamic text, real-time numeric variables, and volumetric capacity bar indicators under varying hospital lighting environments.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
1.3-Inch I2C OLED Display Module	Duino.lk	~LKR 800	128 × 64 resolution, SH1106 driver, I2C communication interface, high-contrast emission.	Best
0.91-Inch I2C OLED Display Module	Duino.lk	~LKR 800	128 × 32 resolution, SSD1306 driver, compact long form-factor, 4-pin I2C hookup.	Too Small
2.42-Inch 128x64 OLED Display Module	Duino.lk	~LKR 10,000	128 × 64 resolution, SSD1309 driver, larger physical viewing space, SPI/I2C selectable.	High Cost

Table 10: Digital Gauge Display Comparison

Justification: The 1.3-Inch I2C OLED display is selected as the ideal interface component. Its self-illuminating pixels ensure complete readability from sharp viewing angles without requiring an active backlight, which is critical for quick operator inspections.

The 0.91-inch alternative is rejected because its limited 128 × 32 resolution cannot fit simultaneous textual logs for both water level meters and chemical capacity tracking. Conversely, the 2.42-inch screen offers an optimal larger viewing area but commands an overly steep budget premium. The 1.3-inch variant balances economic feasibility with sufficient screen real estate to project progress bars representing total volumetric parameters.

4.10 Audio-Visual Emergency Alarm Subsystem

To guarantee facility protection against system abnormalities-such as chemical container exhaustion, unaligned flow deviations, or sensor failures-the system integrates a physical audio-visual signaling device.

Comparative Analysis:

Product	Supplier	Est. Cost	Key Specifications	Verdict
Mini Motor Alarm Siren MS-190	Duino.lk	~LKR 2,400	AC 220V operation, high-speed physical motor siren mechanism, extreme high decibel output.	High Cost
12V DC Electronic Buzzer with Warning Light	Duino.lk	~LKR 990	DC 12V operation, unified structural piezo element with integrated red flashing LED bulb.	Best
Security DC 12V Loud Alarm Siren Horn	Daraz.lk	~LKR 3,200	DC 12V input, 15W horn drive, 115dB output scale, designed for wide outdoor perimeters.	High Cost

Table 11: Alarm Subsystem Component Matrix

Justification: The 12V DC Electronic Buzzer with integrated Warning Light is chosen as the ideal alert element. Because it runs directly on a 12V DC rail, it can be driven safely by the controller using a low-cost logic-level transistor switch, matching the voltage profile of the system’s solenoid valves and pumps.

The MS-190 motor siren is rejected because its 220V AC input introduces severe electromagnetic noise and hazardous high-voltage integration into a fluid-filled instrumentation cabinet. Furthermore, the 115dB outdoor horn is dismissed as its acoustic volume is structurally excessive for an indoor hospital utility room. The selected 12V piezo buzzer provides clear, local audio-visual feedback to operators while remaining highly economical.

4.11 Summary Cost Comparison

Table 12 consolidates the recommended products and their estimated unit costs.

Subsystem	Selected Product	Unit Cost (LKR)	Qty	Line Total (LKR)
Water level – flow sensor	SEN-HZ21WA (HT30)	~3,000	2	~6,000
Hydrostatic level sensing	0–0.5 MPa Pressure Sensor G1/4	~6,990	1	~6,990
Chemical level – load cell	50 kg body load cell	~260	1	~260
Dispensing tank sensor	JSN-SR04T ultrasonic	~1,690	10	~16,900
Chemical line flow meter	YF-S201C G $\frac{1}{2}$	~1,600	1	~1,600
Control valve (per tank)	FPD270A solenoid	~1,200	10	~12,000
Dosing pump	8W 12V peristaltic pump	~3,500	1	~3,500
Master controller	ESP32 Development Board (38-Pin)	~1,490	1	~1,490
Slave controllers	ESP32 Development Board (38-Pin)	~1,490	10	~14,900
Total Estimated Hardware Cost				~63,640

Table 12: Recommended Component Cost Summary

The total estimated hardware cost for the complete system (master controller, 10 slave controllers, and all associated sensors, valves, and pumps for 10 dispensing tanks) is approximately **LKR 63,640**.

5 Component Failures and Automated Fail-Safes

The overall system can be categorised into four localized subsystems:

1. Water Supply Subsystem — Hydrostatic Pressure Level Sensor Failure
2. Water Supply Subsystem — Inline Flow Meter Failure
3. Chemical Dosing Subsystem — Load Cell and Weight Platform Failure
4. Chemical Dosing Subsystem — Actuator Failure (Pump & Solenoid Valves)

By partitioning the system into localized subsystems, the central controller can isolate functional tasks and execute targeted fault-detection routines. If a critical failure occurs within one boundary, the controller can shut down that specific subsystem or safely swap to a computational backup loop without forcing a blind, immediate crash of the entire plant.

5.1 Water Supply Subsystem: Hydrostatic Pressure Level Sensor Failure

The pressure transducer mounted at the base of the 2,000-liter water tank is critical for preventing dry-running or catastrophic overflows.

Potential Failure Modes

1. Mineral buildup or bio-film accumulation blocks the sensor orifice, causing the diaphragm to freeze or misread (sensor drift).
2. Broken signal wires or terminal corrosion leading to full signal loss (0 V) or maximum saturation voltage (5 V).

Automated Detection Strategy

Out-of-Bounds (OOB) Windowing:

Since the calibrated output range of the sensor is strictly 0.5 V (Empty) to 4.5 V (Full), any reading outside this window indicates hardware failure.

Software Implementation:

```
if (voltage < 0.4 || voltage > 4.6) {  
    triggerAlarm("CRITICAL:  
                Water_Pressure_Transducer_Hardware_Fault_Detected!");  
}
```

Fail-Safe Deployments

- **Redundant High-Level Overflow Float Switch:**

A secondary, completely independent mechanical float switch is placed at the absolute top threshold (10.16 m) of the tank. If the pressure sensor falsely reports a low water level due to a clog, the rising water mechanically trips this float switch, creating a hard electrical override that cuts power to the inlet supply valve immediately.

- **Timeout Interlocks:**

If the inlet valve remains open continuously for longer than a calculated maximum duration (e.g., 45 minutes) without the pressure sensor registering an increase in water level, the controller shuts the valve down under an assumed “Sensor Frozen / Pipe Burst” error flag.

5.2 Water Supply Subsystem: Inline Flow Meter Failure

When running the algorithmic flow integration routine, a mechanical or electronic failure in the flow meters can blind the system to the true volume parameters.

Potential Failure Modes

1. **Turbine Rotor Jamming:**

Particulate debris from the main water line jams the internal nylon impeller, causing the sensor to output zero pulses despite fluid actively passing through.

2. **Hall-Effect Sensor Degradation:**

Internal semiconductor degradation due to temperature or manufacturing flaws, resulting in missing pulses or a dead signal line.

Automated Detection Strategy

State Cross-Examination (Valve vs. Sensor):

The controller cross-references the commanded state of the primary inlet/discharge solenoid valves against incoming flow pulses.

Software Implementation:

If the input solenoid valve is commanded HIGH (Open) for more than 5 seconds but `inflowPulses == 0`, a rotor jam or sensor failure is flagged immediately.

```
if (valveState == HIGH && inflowPulses == 0 && elapsed > 5000) {
    triggerAlarm("WARNING:
        Flow_Meter_Rotor_Jam_or_Signal_Fault_Detected!");
}
```

Fail-Safe Deployments

- **Mathematical Cross-Check Verification:**

The controller continuously compares the volume derived from the Hydrostatic Pressure Sensor (V_{pressure}) against the volume derived from the Flow Integration Algorithm ($V_{\text{algorithm}}$). If the absolute divergence between the two measurements exceeds 10% (200 Liters), a diagnostic warning is triggered, and the system defaults exclusively to the pressure sensor while flagging the flow meter for maintenance:

$$|V_{\text{pressure}} - V_{\text{algorithm}}| > 200 \text{ L} \Rightarrow \text{FLOW METER FAULT}$$

5.3 Chemical Dosing Subsystem: Load Cell and Weight Platform Failure

The load cell platform tracking the 10-liter Isopropyl Alcohol (IPA) container is the primary line of defense against running the chemical dosing sequence dry.

Potential Failure Modes

1. Mechanical Shifting / Structural Binding:

The 10 L container shifts off-center, or a surrounding structural component physically touches the platform, creating a “force bridge” that unloads the strain-gauge.

2. HX711 Amplifier Communication Loss:

Data line disconnection (SDA/SCL) between the 24-bit analog-to-digital converter module and the central microcontroller.

Automated Detection Strategy

• Weight Static Noise Detection:

A functional load cell experiences minor micro-gram fluctuations due to nearby pump vibrations. If the load cell readings return an absolute, mathematically static value down to 4 decimal places over a prolonged period, it indicates an ADC communication lockup.

• Negative Mass Validation:

If $M_{\text{current}} < M_{\text{tare}}$ (e.g., reading reports -2 kg), the software recognizes a calibration shift or structural binding error:

$$M_{\text{current}} < M_{\text{tare}} \Rightarrow \text{LOAD CELL BINDING / CALIBRATION FAULT}$$

Fail-Safe Deployments

• Flow Dilution Mirroring:

The Computational Chemical Depletion Model ($\int Q_{\text{chem}} dt$) serves as the immediate fail-safe engine. If the load cell hardware loop drops out or experiences a data fault, the system switches completely to the mathematical algorithm to finish the active batch cycle safely before locking down and requesting an operator inspection.

• E-Stop Default State:

The chemical pump’s motor driver is wired to a Normally Closed (NC) Relay. If the microcontroller experiences a total hardware crash, power to the relay coil drops out, physically breaking the power line to the chemical pump to prevent runaway chemical dosing.

5.4 Chemical Dosing Subsystem: Actuator Failure (Pump & Solenoid Valves)

Mechanical actuators are susceptible to physical wear over extended duty cycles in clinical environments.

Potential Failure Modes

1. Solenoid Valve Spring Fatigue (Stuck Open):

The internal spring fails, keeping the valve open even when power is removed, causing unintended chemical or water siphon leakage.

2. Peristaltic Pump Tubing Rupture:

The silicone tubing cracks under prolonged mechanical stress, resulting in internal chemical leakage, a drop in output pressure, and a total loss of dosing capacity.

Automated Detection Strategy

- **Unintended Flow Accumulation Logging:**

If all system pumps are commanded LOW (Off) and all valves are shut, but the SEN-HZ21WA micro-flow meter continues to register pulses, the controller instantly identifies a “Stuck Open Valve” or a “Siphon Leak”:

`pumpState = LOW, valveState = CLOSED, pulses > 0` $\Rightarrow$ **STUCK VALVE or LEAK**

Fail-Safe Deployments

- **Dual-Gate Isolation (Series Plumbed):**

To eliminate a single point of failure causing chemical contamination, the dosing sequence utilises a secondary safety valve plumbed in series immediately downstream of the main control valve, as illustrated in Figure 10.

Both valves must receive a HIGH command from separate microcontroller GPIO pins to allow fluid to pass. If Valve 1 becomes physically stuck open, Valve 2 acts as a redundant barrier to stop fluid flow completely when the dosing cycle ends.



Figure 1: Dual-Gate Series Valve Isolation Architecture

- **Physical Containment Basin (Bund Wall):**

The 10-liter IPA container and pump assembly sit inside a localized, sealed secondary containment plastic tray. If a tube ruptures, the volatile Isopropyl Alcohol is captured safely within the basin rather than leaking onto the facility floor, eliminating structural pooling and fire hazards.

6 Distributed Data Acquisition and Network Communication Protocol

To establish a robust data highway between the localized dispensing tank sub-controllers (Slave nodes) and the central system hub (Master controller), a resilient communication infrastructure is mandatory. Given the critical nature of hospital utility environments, the network must maintain zero-loss data frames over extended physical distribution distances while operating reliability and electromagnetic interference (EMI).

The communication architecture of this system is partitioned into two clearly defined layers:

1. **Local Control Layer (Slave → Master):** Real-time, deterministic wired communication between all distributed dispensing tank slave nodes and the central master controller via a CAN Bus backbone.
2. **Cloud Reporting Layer (Master → Cloud):** Periodic upstream telemetry from the master controller to a cloud analytics platform via Wi-Fi, enabling hospital management to access consumption reports, refill histories, and system health dashboards remotely.

Comparative Analysis of Local Network Protocols:

Methodology	Physical Layer	Topology	Key Technical Attributes	Verdict
Controller Area Network (CAN Bus)	Twisted-pair copper wiring (Differential signaling)	Multi-Drop Bus (Linear)	High EMI noise immunity; hardware-level error checking and auto-retransmission; max bus distance up to 1 km; deterministic message prioritization via arbitration ID	Selected
MQTT over Wi-Fi	Wireless IEEE 802.11 (Wi-Fi) / TCP-IP	Broker-Client	Lightweight IP protocol; publish-subscribe model; high software abstraction; entirely dependent on RF signal path integrity	Rejected

Table 13: Local Control Layer Communication Protocol Evaluation

6.1 Alternative 1: Wireless MQTT Client-Broker Architecture (Rejected)

The MQTT protocol was rigorously evaluated as a wireless framework for streaming localized telemetry parameters; such as flow pulse counts, load cell mass readings, and dispensing tank trigger states back to the central ESP32 master core. While MQTT offers clean data serialization and straightforward horizontal scalability for adding additional nodes, its dependence on a 2.4 GHz or 5 GHz Wi-Fi radio frequency (RF) infrastructure introduces unacceptable vulnerabilities in a clinical utility setting.

Hospitals feature dense structural compositions, including heavy reinforced concrete load-bearing walls, lead-shielded radiology units, and continuous acoustic paneling, which collectively induce severe RF signal attenuation, and standing wave interfering. Furthermore, if the hospital cannot make sure the wifi connection over the all the places in the hospital premises, this will lead huge connectivity issues with slave to master.

Because no wireless protocol can guarantee the bounded latency and zero-loss frame delivery required for real-time safety-critical control loops in a hospital environment, MQTT is rejected for the local control layer.

6.2 Alternative 2: Controller Area Network (CAN Bus) - Selected

The Controller Area Network (CAN Bus) protocol is selected as the primary network architecture for the local control plant. Originally engineered for automotive powertrain and heavy industrial control systems where deterministic, fault-tolerant communication is non-negotiable, CAN Bus is architecturally aligned with the requirements of this deployment.

Physical Layer: Differential Signaling

CAN Bus transmits data as the voltage difference across two conductors: **CAN_H** (CAN High) and **CAN_L** (CAN Low). Under a dominant (logic 0) bit, CAN_H is driven to +3.5 V while CAN_L is pulled to +1.5 V, producing a differential of +2 V. Under a recessive (logic 1) bit, both lines float to +2.5 V, collapsing the differential to 0 V.

Any external electromagnetic noise radiated by hospital pumps, compressors, or sterilization units strikes both conductor lines simultaneously and with equal magnitude. Because the receiver amplifier measures only the differential voltage and not the absolute line potential, the common-mode noise component cancels entirely — preserving signal integrity regardless of the ambient electromagnetic environment.

Bus Topology and Node Architecture

All ESP32 slave nodes and the ESP32 master node are connected in a linear multi-drop bus configuration using a single twisted-pair cable terminated at both physical ends with 120 Ω resistors to suppress reflective standing waves. Each node interfaces to the physical CAN bus via a dedicated **MCP2515 SPI CAN Controller** paired with a **TJA1050 CAN Bus Transceiver**, providing hardware-level message buffering and automatic error frame retransmission independent of the application firmware.

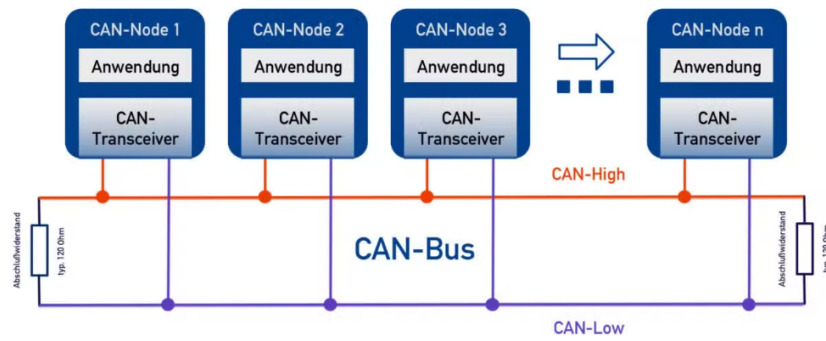


Figure 2: CAN BUS Architecture

CAN Bus Parameter	Value / Description
Physical Medium	Twisted-pair copper cable (shielded recommended in hospital EMI zones)
Signaling Method	Differential voltage (CAN_H and CAN_L)
Maximum Bus Length	Up to 1 km at 50 kbps; up to 40 m at 1 Mbps
Deployed Baud Rate	500 kbps (suitable for hospital corridor runs up to ~100 m)
Maximum Nodes	110 nodes per bus segment
Error Detection	CRC check, ACK verification, bit stuffing, frame check, bus monitoring
Arbitration	Non-destructive bitwise ID arbitration (lower ID = higher priority)
Hardware Interface	MCP2515 SPI CAN Controller + TJA1050 Transceiver per node
Bus Termination	120 Ω resistors at both physical ends of the bus

Table 14: CAN Bus Deployment Parameters

Message Prioritization and Frame Structure

Each slave node is assigned a unique CAN arbitration identifier (ID). Lower numeric IDs carry higher transmission priority, allowing the master to configure the dosing request queue directly in hardware.

- ID 0x001 — Emergency chemical leak alarm (highest priority)
- ID 0x010 — Dispensing tank low-level refill request
- ID 0x020 — Periodic flow meter pulse count telemetry
- ID 0x030 — Load cell mass reading telemetry

- ID 0xFF — Heartbeat / node health ping (lowest priority)

If two slave nodes simultaneously assert a refill request, the node with the lower arbitration ID wins the bus non-destructively, and the higher-ID node automatically defers and retransmits after the current frame completes implementing the first-request-first-serve queue natively at the hardware layer without any software scheduling overhead.

6.3 Cloud Reporting Layer: Master Node Wi-Fi Uplink

While CAN Bus governs all latency-critical, safety-essential communication within the local control plant, the ESP32 master controller independently leverages its onboard **IEEE 802.11 b/g/n Wi-Fi radio** to maintain a non-blocking upstream channel to a cloud analytics platform. This dual-interface architecture isolates real-time control traffic from reporting traffic entirely a failure in the hospital's Wi-Fi infrastructure has zero impact on local CAN Bus dosing operations.

Data Collected and Transmitted

The master controller aggregates the following telemetry streams from all slave nodes over the CAN bus and periodically forwards structured JSON payloads to the cloud endpoint:

Data Stream	Source	Report Generated
Volume dispensed per cycle	Slave node (flow meter)	Chemical usage by dispensing tank location
Refill request timestamps	Slave node (level sensor)	Frequency of refilling per tank per day/week
Cumulative IPA consumed	Master (load cell integration)	Daily and weekly chemical consumption totals
Tank refill queue log	Master (CAN arbitration log)	First-serve queue audit trail
Fault event log	Master (alarm subsystem)	System health and fault frequency report
Water supply tank level	Master (pressure transducer)	Water inventory trend and low-level alerts

Table 15: Cloud Telemetry Data Streams and Generated Reports

Communication Architecture

The master node publishes aggregated telemetry to a cloud MQTT broker (A hospital-hosted MQTT server) using the **MQTT over TLS/SSL** protocol. Payloads are structured as lightweight JSON objects and published on a configurable interval (every 120 seconds for live telemetry, daily for summary reports). Hospital management and maintenance personnel access the data through a cloud dashboard to monitor:

- **Chemical usage by dispensing tank location** : identifying high-consumption zones across the hospital floor plan.

- **Daily and cumulative chemical consumption** : enabling procurement planning and early supply replenishment requests.
- **Frequency of refilling per tank** : detecting abnormal refill rates that may indicate a leaking dispensing tank or a malfunctioning level sensor.
- **System fault history** : providing a maintenance audit trail for regulatory compliance in a clinical environment.

7 System Architecture

The following Architectural diagram depicts how the system works.

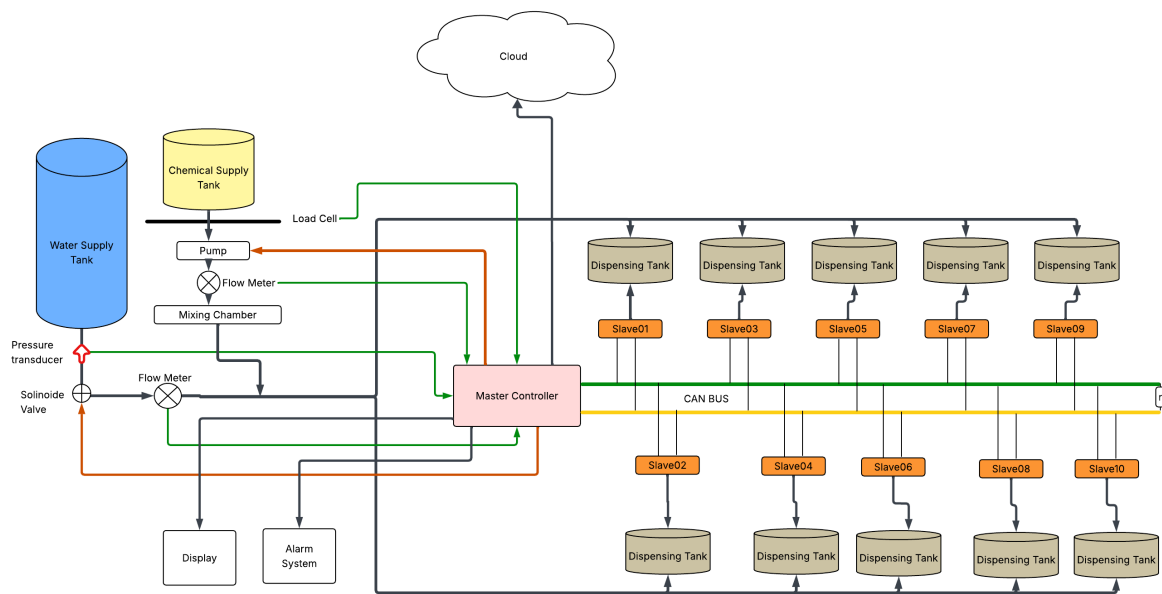


Figure 3: System Architecture Diagram

8 Block Diagram

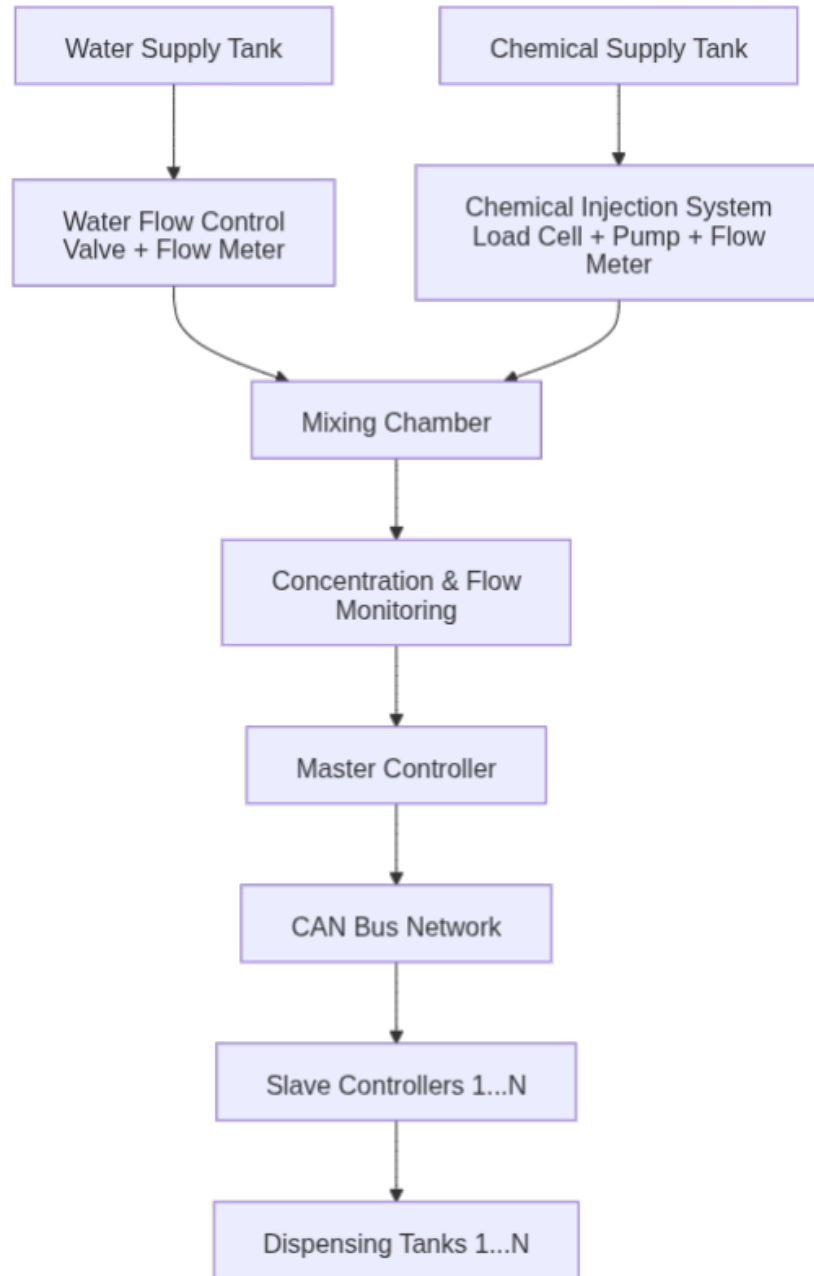


Figure 4: System Block Diagram

9 Operational Algorithm for IOT components

As discussed in Section 4, the overall system is composed of four primary subsystems. For ease of understanding and visualization of the complete system operation, the operational algorithms and flowcharts associated with each subsystem are presented in the following

sections.

Master Controller

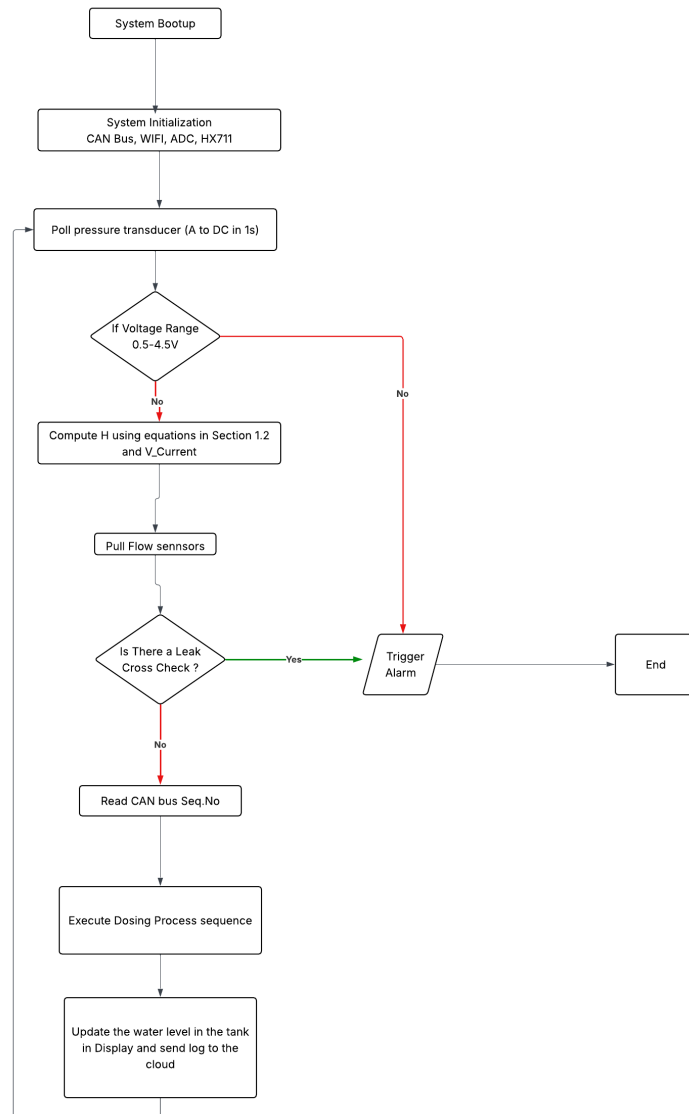


Figure 5: Operation Flow chart of master controller

Slave controller

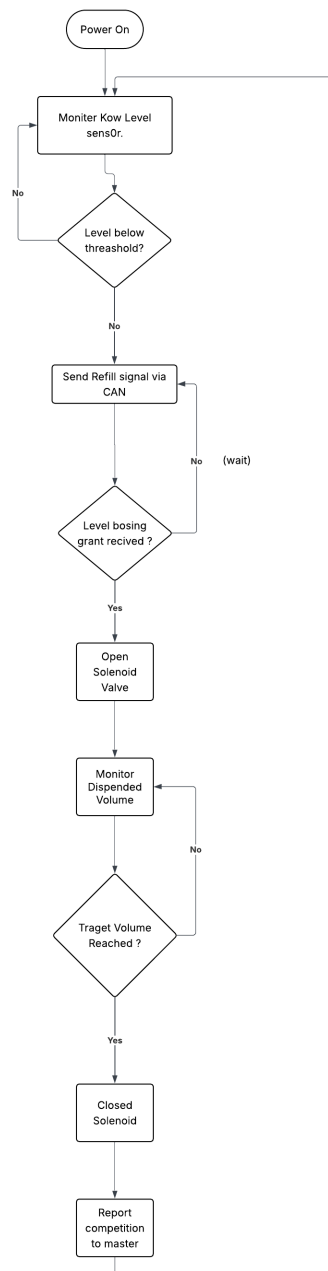


Figure 6: Operation Flow chart of Slave controller

Chemical dosing subsystem

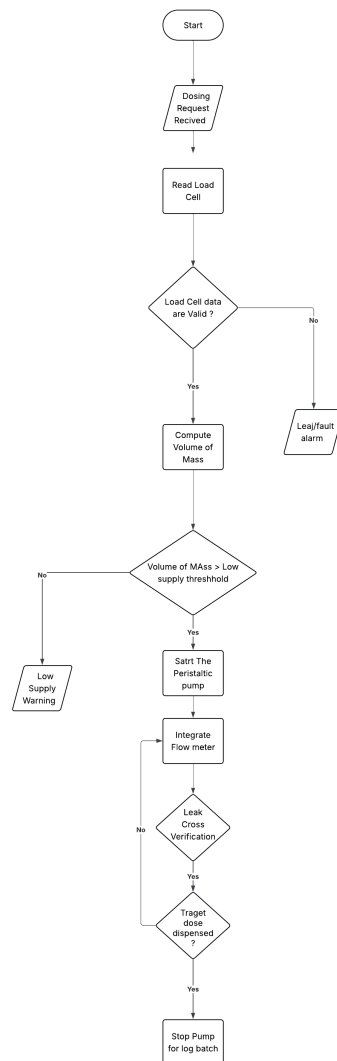


Figure 7: Operation Flow chart of Chemical dosing subsystem

Water level sensing + fault detection

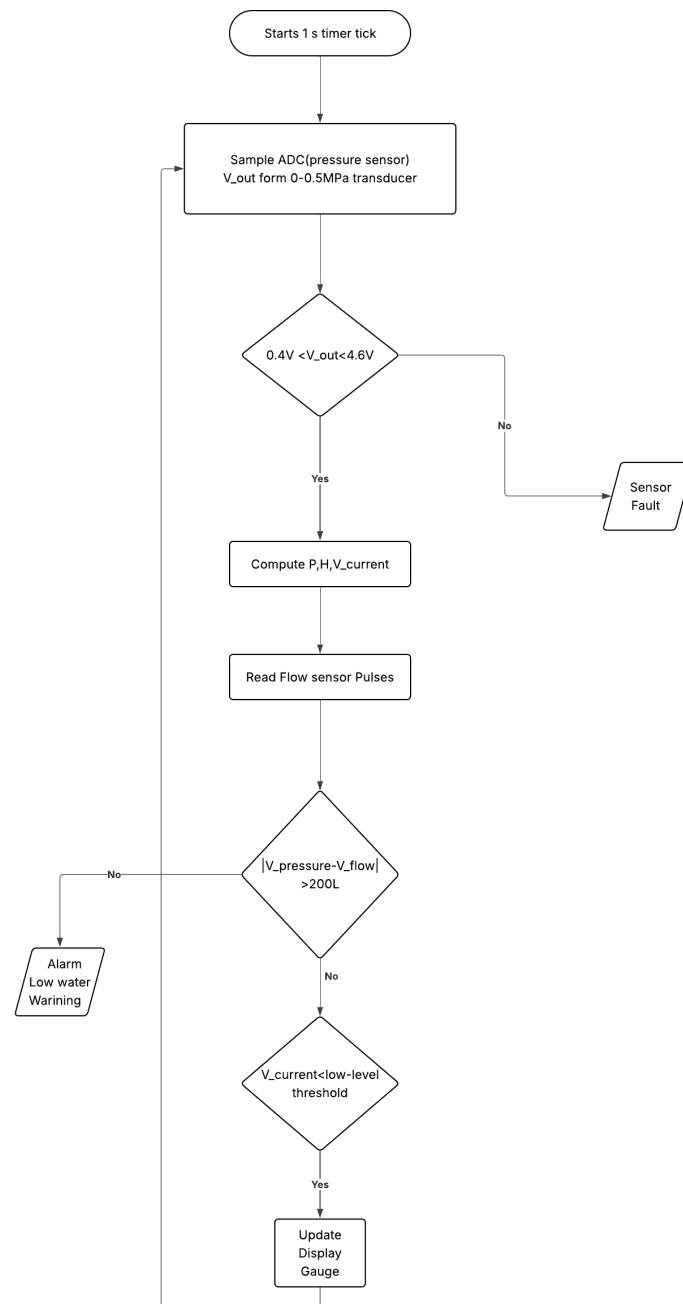


Figure 8: Operation Flow chart of Water level sensing + fault detection

10 Implement simulate the solution (with one dispensing tank)

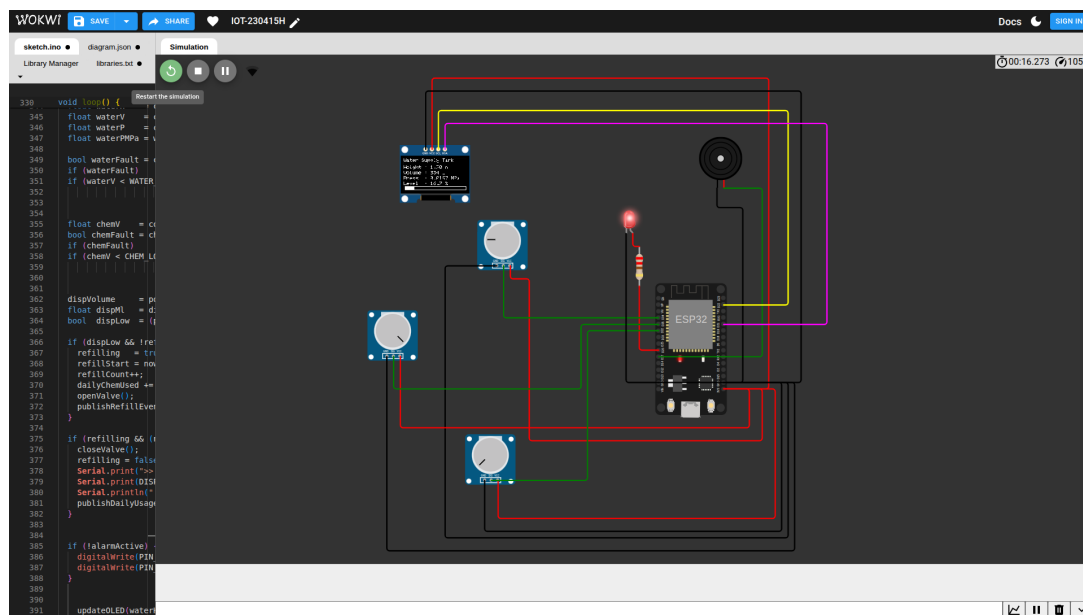


Figure 9: Simulation.

10.1 Documented Code

```
// =====
// Automated Chemical Dosing System      Single Tank Simulation
// Mudaliarachchi NS      230415H
// CS3243 IoT Devices and Applications
//
// POT1 (GPIO34)      simulates 0 0 .5 MPa pressure transducer
// POT2 (GPIO35)      simulates JSN-SR04T dispensing tank sensor
// POT3 (GPIO32)      simulates 50 kg load cell (HX711)
// GPIO26             LED: solenoid valve indicator
// GPIO27             Buzzer: alarm
//
// MQTT topics published:
// dosing/tank/location      tank ID and location string
// dosing/tank/refill_event  each refill: volume + timestamp
// dosing/tank/daily_usage   cumulative daily IPA consumed (L)
// dosing/tank/refill_freq   total refill count this session
// dosing/tank/status        water H, V, P + alarm state
// =====

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <PubSubClient.h>

//OLED
#define SCREEN_WIDTH 128
```

```

#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET
);

// Wi-Fi credentials
// Use wokwi in build
const char* WIFI_SSID      = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

//Mosquitto MQTT broker
const char* MQTT_BROKER    = "broker.hivemq.com";
const int   MQTT_PORT      = 1883;
const char* MQTT_CLIENT_ID = "dosing_tank_01";

//Tank identity
const char* TANK_ID        = "TANK-01";
const char* TANK_LOCATION  = "Ward-B, Corridor-2, Hospital-North";

//MQTT topics
const char* TOPIC_LOCATION = "dosing/tank/location";
const char* TOPIC_REFILL_EVT = "dosing/tank/refill_event";
const char* TOPIC_DAILY     = "dosing/tank/daily_usage";
const char* TOPIC_FREQ      = "dosing/tank/refill_freq";
const char* TOPIC_STATUS    = "dosing/tank/status";

const int PIN_POT_WATER = 34;
const int PIN_POT_DISP  = 35;
const int PIN_POT_CHEM  = 32;
const int PIN_LED_VALVE = 26;
const int PIN_BUZZER    = 27;

// configurations
const float TANK_HEIGHT_MAX = 10.16f;
const float TANK_RADIUS     = 0.25f;
const float RHO_WATER       = 1000.0f;
const float G                = 9.81f;
const float TANK_AREA       = 3.14159f * TANK_RADIUS * TANK_RADIUS;

const float IPA_DENSITY      = 0.785f;
const float CHEM_MAX_MASS    = 7.85f;
const float CHEM_TARE        = 0.425f;
const float CHEM_LOW_WARN    = 1.0f;

const float WATER_LOW_WARN   = 200.0f;
const float DISP_LOW_THRESH  = 0.20f;
const float DISP_REFILL_VOL  = 0.5f;

// State variables
bool valveOpen    = false;
bool alarmActive  = false;
float dispVolume   = 0.75f;

unsigned long refillStart = 0;
bool          refilling   = false;

// Reporting variables

```

```

float          dailyChemUsed  = 0.0f;    // L consumed today
int            refillCount    = 0;        // total refills this session
unsigned long  lastMqttReport = 0;        // last status publish (ms)
const unsigned long MQTT_STATUS_INTERVAL = 10000; // publish status
              every 10 s

// MQTT + Wi-Fi clients
WiFiClient     wifiClient;
PubSubClient   mqttClient(wifiClient);

// helper funcs
float readPot(int pin) {
    return (float)analogRead(pin) / 4095.0f;
}

float computeWaterHeight(float f) { return f * TANK_HEIGHT_MAX; }
float computeWaterVolume(float H) { return H * TANK_AREA * 1000.0f; }
float computeHydrostaticPressure(float H) { return RHO_WATER * G * H; }

float computeChemVolume(float f) {
    float M = f * (CHEM_MAX_MASS + CHEM_TARE) - CHEM_TARE;
    if (M < 0.0f) M = 0.0f;
    return M / IPA_DENSITY;
}

bool checkSensorFault(int pin) {
    int r = analogRead(pin);
    return (r == 0 || r == 4095);
}

//
// OLED display
void updateOLED(float waterH, float waterV, float waterPMPa) {
    float pct = (waterH / TANK_HEIGHT_MAX) * 100.0f;

    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    // Title
    display.setCursor(0, 0);
    display.println("Water Supply Tank");
    display.drawLine(0, 9, 127, 9, SSD1306_WHITE);

    // Numeric readings
    display.setCursor(0, 14);
    display.print("Height:");
    display.print(waterH, 2);
    display.println("um");

    display.setCursor(0, 25);
    display.print("Volume:");
    display.print(waterV, 0);
    display.println("L");

    display.setCursor(0, 36);
    display.print("Press:");
    display.print(waterPMPa, 4);

```

```

display.println("MPa");

// Capacity percentage label
display.setCursor(0, 47);
display.print("Level:");
display.print(pct, 1);
display.println("%");

int barWidth = (int)((pct / 100.0f) * 124.0f);
display.drawRect(2, 56, 124, 8, SSD1306_WHITE);
display.fillRect(2, 56, barWidth, 8, SSD1306_WHITE);

display.display();
}

// Alarm
void triggerAlarm(const char* reason) {
    if (!alarmActive) {
        alarmActive = true;
        Serial.print("!!ALARM:");
        Serial.println(reason);
    }
    digitalWrite(PIN_BUZZER, HIGH);
    digitalWrite(PIN_LED_VALVE, HIGH);
}

void openValve() {
    valveOpen = true;
    digitalWrite(PIN_LED_VALVE, HIGH);
    Serial.println(">>ValveOPEN dosing");
}

void closeValve() {
    valveOpen = false;
    if (!alarmActive) digitalWrite(PIN_LED_VALVE, LOW);
    Serial.println(">>ValveCLOSED done");
}

// Wi-Fi connect
void connectWiFi() {
    Serial.print("ConnectingWiFi");
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    int tries = 0;
    while (WiFi.status() != WL_CONNECTED && tries < 20) {
        delay(500);
        Serial.print(".");
        tries++;
    }
    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("connected");
        Serial.print("IP:");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println("FAILED runningoffline");
    }
}

```

```

//MQTT reconnect
void mqttReconnect() {
  if (WiFi.status() != WL_CONNECTED) return;
  int tries = 0;
  while (!mqttClient.connected() && tries < 3) {
    Serial.print("MQTT_␣connect...");
    if (mqttClient.connect(MQTT_CLIENT_ID)) {
      Serial.println("␣OK");
      // Publish tank identity on connection
      char loc[128];
      snprintf(loc, sizeof(loc),
               "{\"tank_id\":\"%s\", \"location\":\"%s\"}",
               TANK_ID, TANK_LOCATION);
      mqttClient.publish(TOPIC_LOCATION, loc, true); // retained
    } else {
      Serial.print("␣failed␣rc=");
      Serial.println(mqttClient.state());
      delay(2000);
    }
    tries++;
  }
}

//Publish refill
void publishRefillEvent(float volumeL) {
  if (!mqttClient.connected()) return;
  char payload[200];
  snprintf(payload, sizeof(payload),
           "{
            \"tank_id\":\"%s\",
            \"location\":\"%s\",
            \"volume_dispensed_L\":%.3f,
            \"total_refills_session\":%d,
            \"daily_chem_used_L\":%.3f,
            \"uptime_ms\":%lu
           }",
           TANK_ID,
           TANK_LOCATION,
           volumeL,
           refillCount,
           dailyChemUsed,
           millis());
  mqttClient.publish(TOPIC_REFILL_EVT, payload);
  Serial.println("MQTT_␣>>␣refill_event␣published");
}

//Publish periodic status
void publishStatus(float waterH, float waterV, float waterPMPa,
                  float chemV, float dispMl, bool alarm) {
  if (!mqttClient.connected()) return;
  char payload[300];
  snprintf(payload, sizeof(payload),
           "{
            \"tank_id\":\"%s\",
            \"location\":\"%s\",
            \"water_height_m\":%.3f,
            \"water_volume_L\":%.1f,
            \"water_pressure_MPa\":%.4f,

```

```

        "\"chem_remaining_L\":%.3f,\"
        "\"disp_tank_mL\":%.0f,\"
        "\"valve_open\":%s,\"
        "\"alarm_active\":%s,\"
        "\"refill_count\":%d,\"
        "\"daily_chem_used_L\":%.3f\"
    }\",
    TANK_ID,
    TANK_LOCATION,
    waterH, waterV, waterPMPa,
    chemV, dispMl,
    valveOpen ? "true" : "false",
    alarm      ? "true" : "false",
    refillCount,
    dailyChemUsed);
mqttClient.publish(TOPIC_STATUS, payload);
}

//Publish daily usage summary
void publishDailyUsage() {
    if (!mqttClient.connected()) return;
    char payload[150];
    snprintf(payload, sizeof(payload),
        "{
        \"tank_id\": \"%s\",
        \"location\": \"%s\",
        \"daily_chem_used_L\": %.3f,
        \"refill_frequency_per_session\": %d
        }\",
        TANK_ID,
        TANK_LOCATION,
        dailyChemUsed,
        refillCount);
    mqttClient.publish(TOPIC_DAILY, payload, true); // retained
    mqttClient.publish(TOPIC_FREQ, payload, true);
    Serial.println("MQTT>>daily_usage+refill_freqpublished");
}

// Setup
void setup() {
    Serial.begin(115200);

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("SSD1306 failed");
        while (true);
    }
    display.clearDisplay();
    display.display();

    pinMode(PIN_LED_VALVE, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    digitalWrite(PIN_LED_VALVE, LOW);
    digitalWrite(PIN_BUZZER, LOW);

    analogReadResolution(12);
    analogSetAttenuation(ADC_11db);

    connectWiFi();

```

```

mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
mqttClient.setBufferSize(512); // ensure large JSON payloads fit
mqttReconnect();

Serial.println("=====");
Serial.println("AutomatedChemicalDosingSystem Sim v1.1");
Serial.println("Mudaliarachchi NS 230415H CS3243");
Serial.println("=====");
delay(1000);
}

// Main loop
void loop() {
    unsigned long now = millis();
    alarmActive = false;

    // Keep MQTT alive
    if (!mqttClient.connected()) mqttReconnect();
    mqttClient.loop();

    float pot1 = readPot(PIN_POT_WATER);
    float pot2 = readPot(PIN_POT_DISP);
    float pot3 = readPot(PIN_POT_CHEM);

    float waterH = computeWaterHeight(pot1);
    float waterV = computeWaterVolume(waterH);
    float waterP = computeHydrostaticPressure(waterH);
    float waterPMPa = waterP / 1000000.0f;

    bool waterFault = checkSensorFault(PIN_POT_WATER);
    if (waterFault) triggerAlarm("Pressure transducer OOB fault");
    if (waterV < WATER_LOW_WARN && !waterFault)
        triggerAlarm("Water supply LOW below 200L");

    float chemV = computeChemVolume(pot3);
    bool chemFault = checkSensorFault(PIN_POT_CHEM);
    if (chemFault) triggerAlarm("Load cell communication fault");
    if (chemV < CHEM_LOW_WARN && !chemFault)
        triggerAlarm("Chemical supply LOW below 1L");

    dispVolume = pot2 * 0.75f;
    float dispMl = dispVolume * 1000.0f;
    bool dispLow = (pot2 < DISP_LOW_THRESH);

    if (dispLow && !refilling && !alarmActive) {
        refilling = true;
        refillStart = now;
        refillCount++;
        dailyChemUsed += DISP_REFILL_VOL;
    }
}

```

```

    openValve();
    publishRefillEvent(DISP_REFILL_VOL);
}

if (refilling && (now - refillStart >= 5000)) {
    closeValve();
    refilling = false;
    Serial.print(">>_Refill_complete_    _dispensed_");
    Serial.print(DISP_REFILL_VOL, 3);
    Serial.println("_L");
    publishDailyUsage();
}

if (!alarmActive) {
    digitalWrite(PIN_LED_VALVE, valveOpen ? HIGH : LOW);
    digitalWrite(PIN_BUZZER, LOW);
}

updateOLED(waterH, waterV, waterPMPa);

if (now - lastMqttReport >= MQTT_STATUS_INTERVAL) {
    publishStatus(waterH, waterV, waterPMPa, chemV, dispMl, alarmActive);
    lastMqttReport = now;
}

Serial.println("-----");
Serial.print("Water_   H="); Serial.print(waterH,3);
Serial.print("m_   V=");      Serial.print(waterV,1);
Serial.print("L_   P=");      Serial.print(waterPMPa,4);
Serial.println("_MPa");
Serial.print("Chem_   ");      Serial.print(chemV,3);
Serial.println("_L_remaining");
Serial.print("Disp_   ");      Serial.print(dispMl,0);
Serial.print("_mL_");
Serial.println(dispLow ? "LOW" : "OK");
Serial.print("Refills_");      Serial.print(refillCount);
Serial.print("_Daily_");      Serial.print(dailyChemUsed,3);
Serial.println("_L");
Serial.print("Valve=");      Serial.print(valveOpen?"OPEN":"CLOSED");
Serial.print("_Alarm=");      Serial.println(alarmActive?"ACTIVE":"
clear");
Serial.println();

delay(1000);
}

```

Note : Simulation Constraints and Sensor Substitution Rationale

The Wokwi browser-based embedded simulation platform provides a curated hardware library oriented primarily toward common hobbyist and educational components. While it faithfully models the ESP32 microcontroller's peripheral subsystems - including the

12-bit ADC, GPIO interrupts, I2C bus, PWM channels, and Wi-Fi stack — several of the specialised industrial sensors specified in the actual system design are absent from the Wokwi component library.

Real Sensor	Wokwi Substi- tute	GPIO	Functional Gap
0–0.5 MPa pressure transducer	Potentiometer (POT1)	34	No voltage dead-band; OOB fault only partially testable
JSN-SR04T ultrasonic (dispensing tank)	Potentiometer (POT2)	35	Analog poll instead of digital interrupt
HX711 + 50 kg load cell	Potentiometer (POT3)	32	12-bit vs 24-bit resolution; no real-time depletion tracking
SEN-HZ21WA flow sensors ×2	Not simulated	—	Leak detection layer entirely absent
Active buzzer	<code>wokwi-buzzer</code>	27	Fully simulated
Red LED (valve indicator)	<code>wokwi-led</code>	26	Fully simulated
SSD1306 OLED display	<code>wokwi-ssd1306</code>	I2C	Fully simulated
ESP32 microcontroller	<code>wokwi-esp32-dev</code>	—	Fully simulated including Wi-Fi and MQTT

Table 16: Wokwi Sensor Substitution Summary

11 Schematic Diagram

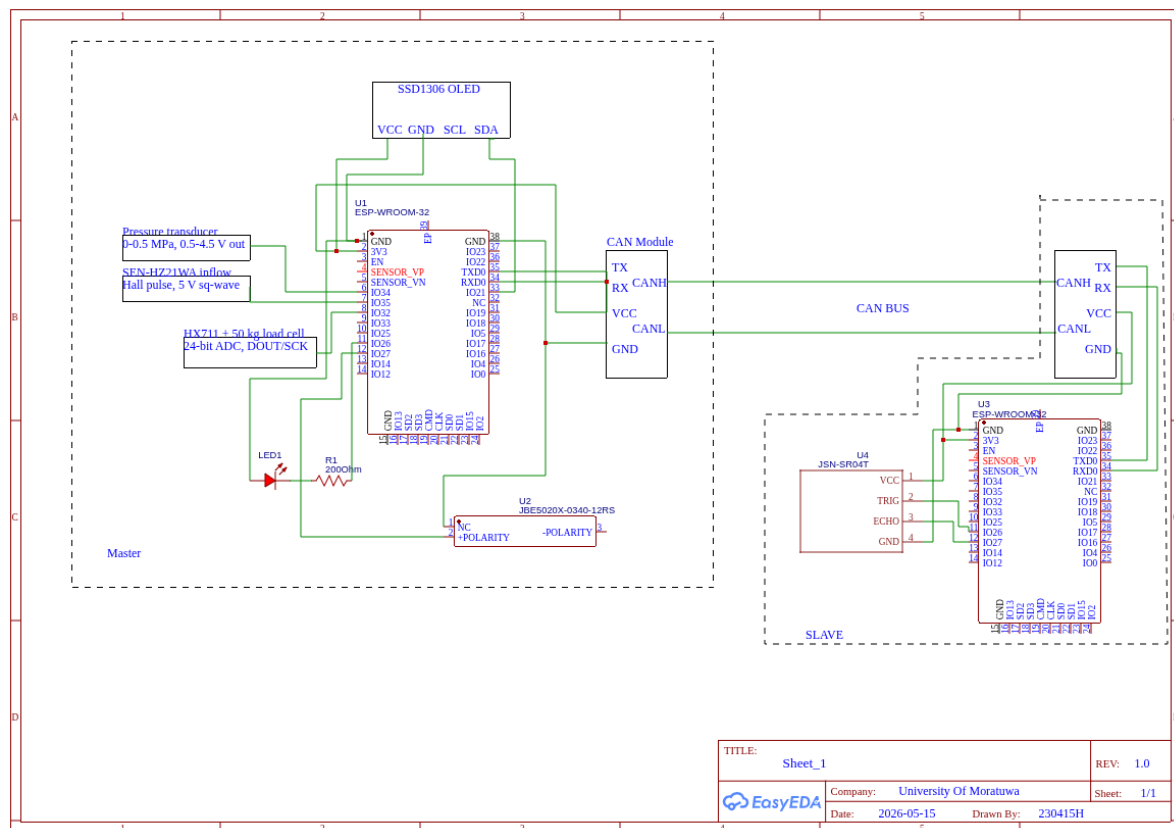


Figure 10: Schematic Diagram With CAN Bus

References

1. *ESP32 CAN Bus Tutorial with MCP2515*. YouTube. Available at: [Click Here](#) [Accessed: May 2026].
2. DFRobot. *Gravity: Non-contact Liquid Level Sensor Tutorial*. DFRobot Wiki. Available at: [Click Here](#) [Accessed: May 2026].
3. WonderfulPCB. *Comparing Popular Sensor Brands: Prices, Value and Features*. WonderfulPCB Blog. Available at: [Click Here](#) [Accessed: May 2026].